

Computational Physics

Solving partial differential equations based on preconditioning-pretraining physics-informed neural network

Zihan Wang^a, Ziyue Hu^a, Mingwei Yang^a, Yalin Dong^b, Wenlong Huang^{a, ID, *}, Haijun Ren^{c, **}^a School of Computer Science and Technology, Anhui Engineering Laboratory for Industrial Internet Intelligent Applications and Security, Anhui University of Technology, Ma'anshan, Anhui 243002, China^b Department of Mechanical Engineering, The University of Akron, Akron, OH 44325, USA^c CAS Key Laboratory of Geospace Environment and Department of Engineering and Applied Physics, School of Physical Sciences, University of Science and Technology of China, Hefei, Anhui 230026, China

ARTICLE INFO

The review of this paper was arranged by Prof. Andrew Hazel

Dataset link: <https://github.com/huangwenlongplasma/PP-PINN>

Keywords:

Physics informed neural network
High frequency solution
Strong nonlinearity
Preconditioning-pretraining

ABSTRACT

Physics-Informed Neural Network (PINN) is a deep learning framework that has been widely employed to solve spatial-temporal partial differential equations (PDEs) across various fields. However, recent numerical experiments indicate that the vanilla-PINN often struggles with PDEs featuring high-frequency solutions or strong nonlinearity. To enhance PINN's performance, we propose a novel strategy called the Preconditioning-Pretraining Physics-Informed Neural Network (PP-PINN). This approach involves transforming the original task into a new system characterized by low frequency and weak nonlinearity over an extended time scale. The transformed PDEs are then solved using a pretraining approach. Additionally, we introduce a new constraint termed "fixed point", which is beneficial for scenarios with extremely high frequency or strong nonlinearity. To demonstrate the efficacy of our method, we apply the newly developed strategy to three different equations, achieving improved accuracy and reduced computational costs compared to previous approaches which incorporate the pretraining technique. The effectiveness and interpretability of our PP-PINN are also discussed, emphasizing its advantages in tackling high-frequency solutions and strong nonlinearity, thereby offering insights into its broader applicability in complex mathematical modeling.

1. Introduction

Partial differential equations (PDEs) are widely used to describe phenomena in different fields such as physics, chemistry, and material science. Typically, PDEs cannot be solved analytically due to challenges such as strong nonlinearity and complex geometries. Since the 20th century, various numerical methods, such as finite difference, finite element, and finite volume, have been developed to solve PDEs in science and engineering. In these approaches, time-advancing schemes, spatial discretization, and (nonlinear) iteration algorithms must be carefully designed to obtain credible solutions. Furthermore, one will encounter

difficulties when conventional numerical methods are adopted to solve high-dimensional PDEs [1–3].

Recently, a series of deep learning-based frameworks [4,5], such as the physics-informed neural network (PINN) [6], deep Ritz method (DRM) [7], and deep Galerkin method (DGM) [8], have been developed to solve PDEs. As a new forward and backward PDE solver, PINN was first proposed by Raissi et al. in 2019 [6]. Notably, this deep learning-based approach is mesh-free, thus mitigating the curse of dimensionality to a certain extent [6,9]. Furthermore, supervised data included in the training procedure further improve the performance of PINN. Due to its promising potential, PINN and its variants [10–12] have been widely introduced and implemented to solve governing equations in various

* Corresponding author at: School of Computer Science and Technology, Anhui Engineering Laboratory for Industrial Internet Intelligent Applications and Security, Anhui University of Technology, Ma'anshan, Anhui 243002, China.

** Corresponding author.

E-mail addresses: whuang@ahut.edu.cn (W. Huang), hjren@ustc.edu.cn (H. Ren).

<https://doi.org/10.1016/j.cpc.2025.109601>

Received 30 August 2024; Received in revised form 14 March 2025; Accepted 29 March 2025

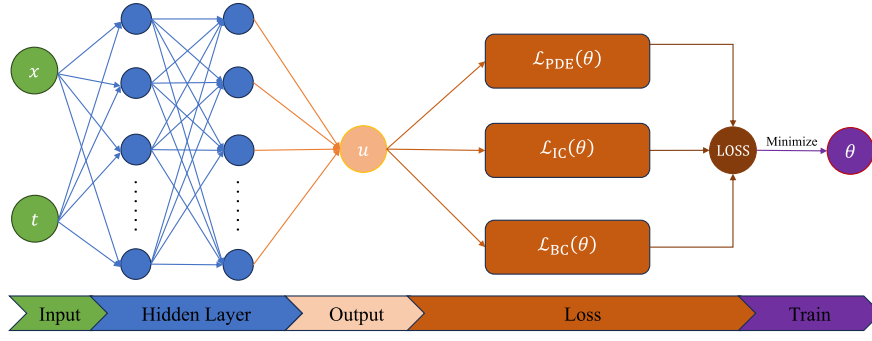


Fig. 1. Schematic diagram of the vanilla-PINN.

fields [13–16]. For example, using PINN, the Boltzmann equations is solved to simulate thermal transport in metal and semiconductor systems [17]. This PDE solver is also used to predict solutions of stiff chemical kinetics in atmospheric chemistry [18] and measure mass transport to the rotating disk electrode [19].

Usually, PDEs are time-dependent. Although PINN has achieved various successes in scientific computation, several difficulties arise when it is used to solve spatial-temporal equations. For instance, because PDEs at different times are trained simultaneously, causality may be violated [20]. On the other hand, capturing high-frequency solutions via PINN is challenging [21]. PINN also fails in cases with strong nonlinearity [22]. To mitigate these difficulties, several methods, such as curriculum training, backward-compatible strategies, pretraining, and causal-respected strategies, have been proposed and implemented [20,22–24]. In a wide parameter regime, these approaches can predict credible solutions. However, the above-mentioned issues have not been fully resolved yet.

In this work, a new strategy named the Preconditioning-Pretraining Physics-Informed Neural Network (PP-PINN) is proposed to further improve the performance of PINN. First, we transform the original PDEs with high-frequency solutions and strong nonlinearity into ones with low-frequency solutions and weak nonlinearity over a long timescale. Then, the new system can be solved via the pretraining strategy. It should be noted that, with appropriately chosen schemes of normalization and domain decomposition, the transformed system in the first sub-domain can be solved stably. Similar to previous work [24], solutions obtained in the sub-domains provide initialization for the neural network and additional supervised data for relatively large sub-domains. Then, the original task in a series of time domains can be solved one by one. To further improve the accuracy of PP-PINN, we also introduce a new constraint termed “fixed point”, which is beneficial for cases with extremely high-frequency solutions or strong nonlinearity. Our new method has the potential for implementation in realistic studies in science and engineering.

Notably, we focus on evaluating the effectiveness of our preconditioning-pretraining strategy for time-dependent systems. For simplicity, all examples presented herein are confined to a one-dimensional (1D) setting. However, we believe that our approach can be seamlessly extended to two-dimensional (2D) and three-dimensional (3D) problems without encountering fundamental difficulties. Furthermore, successfully solving 1D problems will provide valuable insights and experiences that can be applied to higher-dimensional tasks.

The rest of this paper is organized as follows. In Section 2, the architecture of the vanilla-PINN is introduced. We then propose the algorithm of PP-PINN in Section 3. In Section 4, numerical experiments are presented to demonstrate the performance of our method. Finally, we present a summary and discussion in Section 5.

2. Architecture of the Vanilla-PINN

In this section, we introduce the architecture of the vanilla-PINN. As an illustrative example, we solve a one-dimensional convection equation with periodic boundary conditions. Without loss of generality, the unsolved system is presented as follows

$$\frac{\partial u}{\partial t} + \beta \frac{\partial u}{\partial x} = 0, \quad (1)$$

$$u(x, 0) = \alpha \sin(x), \quad (2)$$

$$u(0, t) = u(2\pi, t), \quad (3)$$

where $x \in [0, 2\pi]$, $t \in [0, T]$, and α and β represent the amplitude of the initial condition and the convection coefficient, respectively. Note that u satisfies

$$u(x, t) = \alpha \sin(x - \beta t). \quad (4)$$

On the other hand, the loss function $\mathcal{L}(\theta)$ is typically expressed as [6]

$$\mathcal{L}(\theta) = \omega_{\text{PDE}} \mathcal{L}_{\text{PDE}}(\theta) + \omega_{\text{IC}} \mathcal{L}_{\text{IC}}(\theta) + \omega_{\text{BC}} \mathcal{L}_{\text{BC}}(\theta), \quad (5)$$

where θ represents the hyperparameters of the PINN. $\mathcal{L}_{\text{PDE}}(\theta)$ (ω_{PDE}), $\mathcal{L}_{\text{IC}}(\theta)$ (ω_{IC}), and $\mathcal{L}_{\text{BC}}(\theta)$ (ω_{BC}) are the losses (weights) associated with the PDE, initial condition, and boundary condition, respectively. These are defined as follows

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_i N_j} \sum_{i=1}^{N_i} \sum_{j=1}^{N_j} \left| \frac{\partial u_\theta}{\partial t}(x_i, t_j) + \beta \frac{\partial u_\theta}{\partial x}(x_i, t_j) \right|^2, \quad (6)$$

$$\mathcal{L}_{\text{IC}}(\theta) = \frac{1}{N_i} \sum_{i=1}^{N_i} |u_\theta(x_i, 0) - \alpha \sin(x_i)|^2, \quad (7)$$

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N_j} \sum_{j=1}^{N_j} |u_\theta(0, t_j) - u_\theta(2\pi, t_j)|^2. \quad (8)$$

Here, u_θ represents the predicted solution of the PINN, and N_i and N_j are the number of sampling points in space and time, respectively. It is important to note that the influence of hard constraints has not been examined in our numerical experiments, despite their significance for training PINNs [25,26]. In future work, we plan to extend our research and assess its impact on our PP-PINN.

To obtain the predicted solution u_θ , the loss function $\mathcal{L}(\theta)$ is minimized by training the neural network. The schematic diagram of the vanilla-PINN is shown in Fig. 1.

Similar to the previous work [24], the $L_{2\text{-norm}}$ relative error is used to measure the accuracy of the predicted solution, expressed as

$$L_{2\text{-norm}} = \frac{\sqrt{\sum_{i=1}^{N_i} \sum_{j=1}^{N_j} |u - u_\theta(x_i, t_j)|^2}}{\sqrt{\sum_{i=1}^{N_i} \sum_{j=1}^{N_j} |u(x_i, t_j)|^2}}. \quad (9)$$

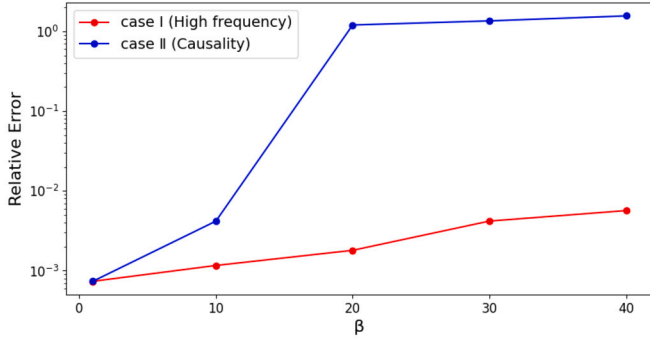


Fig. 2. Non-equivalence of two cases in the vanilla-PINN. The red and blue curves represent case (I) and case (II), respectively. Physical parameters used here are: $T = 1$ and $\alpha = 1$. Hyper parameters used here are as follows. The activation function is tanh, the numbers of hidden layers and neurons are 7 and 60, respectively; the number of sampling points is $N_i = N_j = 1000$; the learning rates for Adam and L-BFGS are 1×10^{-3} and 1; and the weights are $\omega_{\text{PDE}} = \omega_{\text{IC}} = \omega_{\text{BC}} = 1$. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

3. The Algorithm of PP-PINN

As pointed out previously in Refs. [21,23], the vanilla-PINN may face failures as the convection coefficient β increases. To address the challenges posed by high-frequency solutions and strong non-linearity, various methods such as backward-compatible physics-informed neural network (BC-PINN) and pretraining physics-informed neural network (PT-PINN) have been proposed [22,24]. However, these challenges have not yet been completely overcome. In this section, we propose a new strategy named PP-PINN to further improve the performance of PINN.

3.1. A short review of BC-PINN and PT-PINN

In this subsection, we briefly review BC-PINN and PT-PINN. To mitigate strong nonlinearity, a sequential strategy, known as BC-PINN, was proposed in 2022 by Mattheya and Ghosh [22]. In BC-PINN, the entire time domain $[0, T]$ is divided into a series of sub-domains: $[0, T_1]$, $[T_1, T_2]$, ..., $[T_{n-1}, T_n]$, where $0 < T_1 < T_2 < \dots < T_n = T$. The solution of the system at $t = T_k$ is used as the initial condition for the $(k+1)$ th sub-domain, allowing for the sequential solution of PDEs. Furthermore, when solving PDEs within the interval $[T_k, T_{k+1}]$, solutions obtained in previous steps ($[0, T_k]$) are implemented by penalizing deviations from these previously obtained solutions. This is the reason why it is named as “backward compatible”.

Notably, two important ideas have been proposed and implemented in Ref. [22]. First, the trained neural network (NN) within the sub-domain $[T_{k-1}, T_k]$ serves as an initialization for the subsequent interval $[T_k, T_{k+1}]$. This approach significantly enhances the accuracy and efficiency of PINN, particularly for systems exhibiting strong nonlinearity. Second, the importance of the initial condition is addressed in BC-PINN, which has inspired our insight into the “fixed point” constraint.

In contrast to Ref. [22], Guo et al. [24] divided the entire time domain $[0, T]$ into sub-domains as follows: $[0, T_1]$, $[0, T_2]$, ..., $[0, T_n]$, where $0 < T_1 < T_2 < \dots < T_n = T$. And then, PDEs can be solved in these sub-domains. The predicted solutions in smaller sub-domains ($[0, T_k]$) provide a warm start (i.e., initialization of the neural network) and additional data constraints for problems in larger sub-domains ($[0, T_{k+1}]$). Therefore, such a strategy is termed “pretraining”. As shown in Ref. [24], both initialization from pretraining and additional data constraints are crucial for the performance of PT-PINN. Furthermore, several techniques such as resampling method and combination with existing optimizers are adopted in their work, which can further enhance the accuracy and efficiency of PINN based methods. Strategies such as BC-PINN and PT-PINN significantly mitigate the issues associated with high-frequency solutions and strong nonlinearity.

3.2. Two observations in the Vanilla-PINN

Before introducing the main idea of PP-PINN, it is helpful to discuss two phenomena observed in the vanilla-PINN. Without loss of generality, these observations are presented by solving the convection equation using the vanilla-PINN. First, it is known that the vanilla-PINN may fail when the convection coefficient β is large enough due to the high-frequency curse [23]. To address this issue, strategies like BC-PINN and PT-PINN have been proposed.

Mathematically, the problem of PDEs with high-frequency solutions can be transformed into one with a lower β over a longer timescale. Then, the high-frequency curse can be theoretically removed. For example, by defining $t = c\hat{t}$, the convection equation can be rewritten as

$$\frac{\partial u}{\partial \hat{t}} + c\beta \frac{\partial u}{\partial x} = 0, \quad (10)$$

$$u(x, 0) = \alpha \sin(x), \quad (11)$$

$$u(0, \hat{t}) = u(2\pi, \hat{t}), \quad (12)$$

where $\hat{t} \in [0, \hat{T}]$ and $\hat{T} = T/c$. The effective convection coefficient is now $\beta^* = c\beta$. Since c can be chosen to much less than unity, the high-frequency curse is alleviated. However, causality violations may occur when the transformed system is solved by using the vanilla-PINN [20].

To compare the performance of these two approaches, we solve Eq. (10) under two cases: (I) $c = 1$ and (II) $c\beta = 1$. Mathematically, the two cases are equivalent. However, when solved using the vanilla-PINN, the equivalence breaks down. As shown in Fig. 2, the relative error (RE) for both cases is similar when $\beta = 1$. As β increases, the accuracy of case (I) surpasses that of case (II). Specifically, case (I) can be stably solved using the vanilla-PINN in the regime where $\beta \leq 40$, whereas for case (II), RE approaches 100% when $\beta \sim 20$. Consequently, the aforementioned equivalence is broken when solved using the vanilla-PINN. We acknowledge that a comprehensive explanation for the breakdown of equivalence between case (I) and case (II) is beyond our current scope. Instead, we offer a phenomenological interpretation in our study as follows: for case (I), the difficulty arises from the high-frequency curse, whereas for case (II), causality violations lead to failure. Therefore, systems with long timescales are more challenging to solve than those with high-frequency solutions. Fortunately, long-time integration can be effectively managed with the pretraining strategy [24], which will be presented in the next section.

Another important observation concerns the amplitude of the solution for the unsolved equation. In previous works, the solution u is typically of the order $\mathcal{O}(1)$. However, in realistic physical environments, u may differ significantly from unity [27,28]. To study the performance of the vanilla-PINN in such regimes, we solve Eq. (1) with $\alpha = 10^4$ and 10^{-4} .

As shown in Fig. 3, the vanilla-PINN fails to accurately predict solutions for both $\alpha = 10^4$ and 10^{-4} . The REs are 759% and 27%, respectively. This can be qualitatively explained as follows: because $\mathcal{L}(\theta) \propto \alpha^2$, the loss function deviates significantly from unity when $\alpha \gg 1$ or $\alpha \ll 1$. In this parameter regime, the vanilla-PINN cannot be well trained. This results in gradient explosion or vanishing issues. We will later demonstrate that this problem becomes more pronounced in nonlinear systems.

3.3. The main idea of PP-PINN

As discussed above, the vanilla-PINN struggles with PDEs characterized by high-frequency solutions or strong nonlinearity. Problems persist despite that various strategies have been proposed to mitigate these challenges [22,24]. To further improve the accuracy of PDE solvers, we propose a new strategy called the PP-PINN. In PP-PINN, we first transform the original task with high-frequency solutions or strong nonlinearity into a system with low-frequency solutions or weak nonlinearity over a longer timescale. The task of long-time integration is then solved using the pretraining strategy.

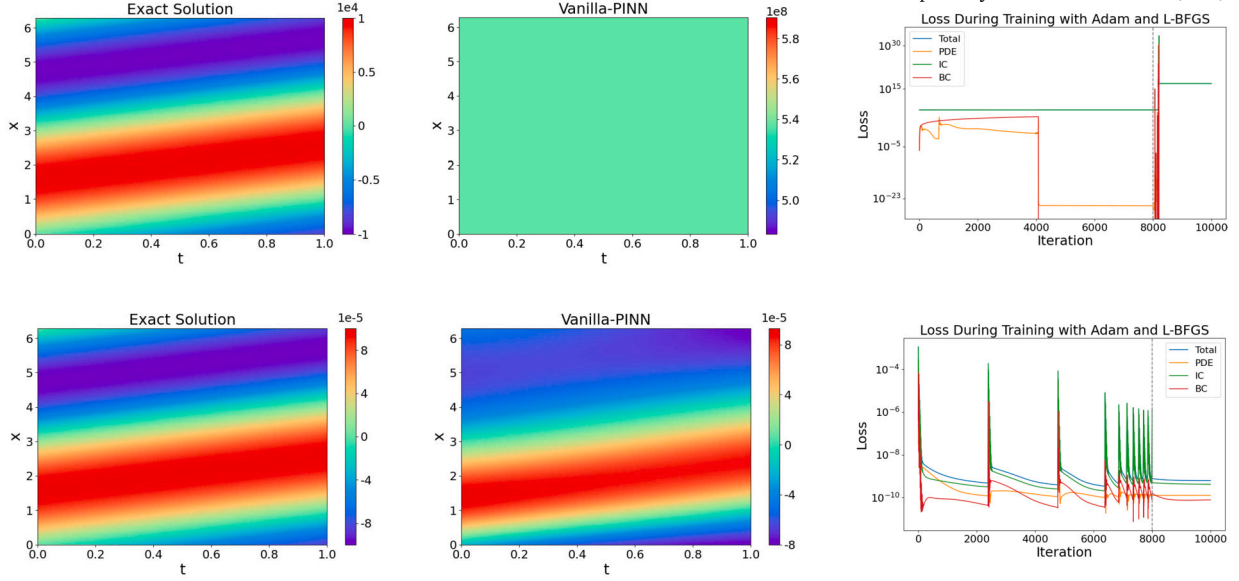


Fig. 3. Exact and predicted solutions of Eq. (1), as well as the training loss of the PINN for $\alpha = 10^4$ (upper) and $\alpha = 10^{-4}$ (lower). Physical parameters used here are $\beta = 1$ and $T = 1$. Hyperparameters are the same as in Fig. 2.

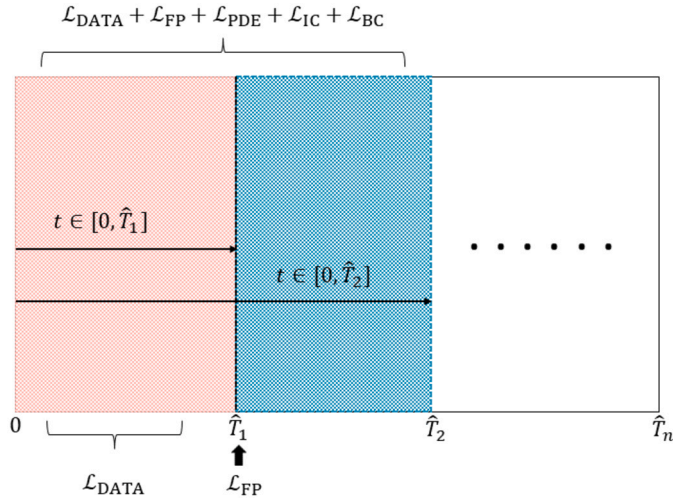


Fig. 4. Schematic diagram of PP-PINN.

To illustrate the procedure of PP-PINN, we present a schematic diagram of this strategy in Fig. 4. As shown in Fig. 4, the entire time domain $[0, \hat{T}]$ is divided into $[0, \hat{T}_1]$, $[0, \hat{T}_2]$, ..., $[0, \hat{T}_n]$, where $\hat{T}_n = \hat{T} = \frac{T}{c}$. The normalization and domain decomposition schemes should be appropriately chosen. When c and $\hat{T}_1$ are properly selected, the transformed equations in $[0, \hat{T}_1]$ can be solved with high accuracy using PINN. The predicted solution in $[0, \hat{T}_1]$ provides initialization for the neural network and additional data constraints for $[0, \hat{T}_2]$, similar to PT-PINN. This approach allows the transformed equations to be solved in sub-domains sequentially.

It is important to note that, at present, we lack a universal principle for selecting normalization and domain decomposition schemes. In this study, our domain decomposition approach aligns with that of PT-PINN [24]. Meanwhile, the choice of normalization scheme remains based on empirical rules. In the future, we will conduct further research on this crucial issue and endeavor to propose a guideline for the selection of normalization and domain decomposition schemes.

To further enhance the accuracy of PP-PINN, we introduce a new data constraint similar to the one in BC-PINN [22]. The solution at $\hat{t} = \hat{T}_k$ can be considered the “initial condition” for the task in $[\hat{T}_k, \hat{T}_{k+1}]$, where $k = 1, 2, \dots$. This constraint is essential for the accuracy of BC-PINN. Although spatial-temporal PDEs are advanced from $[0, \hat{T}_k]$ to $[0, \hat{T}_{k+1}]$, we still set the predicted solution at $\hat{t} = \hat{T}_k$ as an additional data constraint. We will show that this constraint becomes crucial when $\hat{T}_k$ is large.

To clarify, we refer to the predicted solution at $\hat{t} = \hat{T}_k$ as the “fixed point” (FP). For the $(k + 1)$ th sub-domain, the loss function is expressed as

$$\mathcal{L}(\theta) = \omega_{\text{PDE}} \mathcal{L}_{\text{PDE}}(\theta) + \omega_{\text{IC}} \mathcal{L}_{\text{IC}}(\theta) + \omega_{\text{BC}} \mathcal{L}_{\text{BC}}(\theta) + \omega_{\text{DATA}} \mathcal{L}_{\text{DATA}}(\theta) + \omega_{\text{FP}} \mathcal{L}_{\text{FP}}(\theta), \quad (13)$$

$$\mathcal{L}_{\text{DATA}}(\theta) = \frac{1}{N_i N_j} \sum_{i=1}^{N_i} \sum_{j=1}^{N_j} |u_\theta(x_i, \hat{t}_{k,j}) - u_k(x_i, \hat{t}_{k,j})|^2, \quad (14)$$

$$\mathcal{L}_{\text{FP}}(\theta) = \frac{1}{N_i} \sum_{i=1}^{N_i} |u_\theta(x_i, \hat{T}_k) - u_k(x_i, \hat{T}_k)|^2, \quad (15)$$

where $\hat{t}_{k,j} \in [0, \hat{T}_k]$ and u_k is the predicted solution in the k th step. $\mathcal{L}_{\text{DATA}}(\theta)$ (ω_{DATA}) and $\mathcal{L}_{\text{FP}}(\theta)$ (ω_{FP}) are the losses (weights) associated with data and FP, respectively.

In the above setting, $\mathcal{L}_{\text{DATA}}$ requires the same sampling points as $\mathcal{L}_{\text{PDE}}$. During the pretrained step ($[0, \hat{T}_k]$), there are $N_i \times N_j$ sampling points for $\mathcal{L}_{\text{PDE}}$. For simplicity, we adopt the pretrained solution at these sampling points as data constraints for the subsequent training step ($[0, \hat{T}_{k+1}]$). Therefore, in this next training step ($[0, \hat{T}_{k+1}]$), the number of sampling points for $\mathcal{L}_{\text{DATA}}$ is $N_i \times N_j$. Notably, the number of sampling points for $\mathcal{L}_{\text{PDE}}$ in such a step is also set to $N_i \times N_j$. Consequently, the number of sampling points for $\mathcal{L}_{\text{DATA}}$ and $\mathcal{L}_{\text{PDE}}$ remains consistent, although it may not be strictly necessary.

Finally, we outline the algorithm and settings for PP-PINN. The algorithm is summarized as follows

Algorithm 1 PP-PINN.

```

1: Preconditioning: Transform PDEs and time domain appropriately.
2: Input:
3: Architecture of neural network (deep neural network);
4: Split the entire time domain  $[0, \hat{T}_n]$  into  $[0, \hat{T}_1], [0, \hat{T}_2], \dots, [0, \hat{T}_n]$ ;
5: Pretraining step:
6: for  $k = 1, \dots, n-1$  do
7:   if  $k == 1$  then
8:     Minimize the loss function (Eq. (5)) using the combination of Adam
       and L-BFGS;
9:     Save data and parameters of NN ( $\theta$ ) in  $[0, \hat{T}_1]$ ;
10:    Sample data at  $\hat{t} = \hat{T}_1$  as the FP constraint.
11:   else if  $k > 1$  and  $k \leq n-1$  then
12:      $\mathcal{L}(\theta) = \omega_{\text{PDE}} \mathcal{L}_{\text{PDE}}(\theta) + \omega_{\text{IC}} \mathcal{L}_{\text{IC}}(\theta) + \omega_{\text{BC}} \mathcal{L}_{\text{BC}}(\theta) + \omega_{\text{DATA}} \mathcal{L}_{\text{DATA}}(\theta) +$ 
        $\omega_{\text{FP}} \mathcal{L}_{\text{FP}}(\theta)$ ;
13:     Use  $\theta$  obtained in  $[0, \hat{T}_{k-1}]$  as a warm start and train loss functions with
       data and FP constraints obtained in  $[0, \hat{T}_{k-1}]$ ;
14:     Save data and  $\theta$  in  $[0, \hat{T}_{k-1}]$ ;
15:     Sample data at  $\hat{t} = \hat{T}_{k-1}$  as the FP constraint.
16:   end if
17: end for
18: Formal training step:
19: Use  $\theta$  obtained in  $[0, \hat{T}_{n-1}]$  as a warm start and train  $\mathcal{L}(\theta)$  with data and FP
   constraints obtained in  $[0, \hat{T}_{n-1}]$ .

```

In this work, all calculations are implemented using the deep learning framework PyTorch version 2.0 with single precision (float32). All experiments are conducted on a deep neural network (DNN) using NVIDIA GeForce RTX 3090 (24 GB). The combination of Adam and L-BFGS is adopted as described in Ref. [24]. The learning rates for these two optimizers are set to 1×10^{-3} and 1, respectively; the number of iteration steps for Adam and L-BFGS are adjusted to ensure the convergence of predicted solutions. Additionally, all weights in Eq. (13), such as ω_{PDE} , are set to 1.

In fact, we have also studied the influence of weights in our numerical experiments. The weights of the loss terms can significantly enhance the performance of our PP-PINN. However, these weights are currently chosen arbitrarily. The results thus lack interpretability and deviate from the main focus of this work. Consequently, we do not include these results. In future research, we plan to use adaptive weights to further improve the performance of our PP-PINN, which will be presented in a separate publication.

4. Numerical results of PP-PINN

In this section, we solve three different PDEs with high-frequency solutions or strong nonlinearity using our new strategy, PP-PINN. We also compare the results with those obtained using PT-PINN. Numerical results demonstrate that our strategy offers better accuracy than the previous method, while simultaneously reducing computational costs compared to PT-PINN.

4.1. The convection equation

In Sec. 3.3, we introduced a new strategy named PP-PINN. In the approach of PP-PINN, Eq. (1) is transformed into a new form (Eq. (10)). Here, we solve the transformed equation in sub-domains using the pretraining method. To demonstrate the performance of PP-PINN (as outlined in Algorithm 1), we solve the convection equation with $\beta = 200$ and $T = 1$. By setting $c = 0.1$, we obtain $\hat{T} = 10$ and $c\beta = 20$. Following Ref. [24], the entire domain $\hat{t} \in [0, \hat{T}]$ is split into sub-domains: $[0, \hat{T}_1], [0, \hat{T}_2], \dots, [0, \hat{T}_{10}]$, where $\hat{T}_k = k$. Notes that, the convection equation with $c\beta = 20$ and $\hat{T}_1 = 1$ can be stably predicted using the vanilla-PINN. And then, by employing the pretraining strategy, the transformed system in sub-domains can be solved sequentially.

First, we test the performance of PP-PINN without the FP constraint. As shown in Table 1, RE increases with $\hat{T}_k$. This is reasonable since RE

accumulates during calculations. Before $\hat{T}_k = 8$, RE is less than or about 2%. However, when $\hat{T}_k = 8$, RE rises to approximately 28%, indicating a failure at this step. To improve the performance of PP-PINN, additional techniques should be introduced, which will be discussed later.

For comparison, we also solve the convection equation using PT-PINN with $\beta = 200$ and $T = 1$. The entire time domain is uniformly divided into 10 sub-domains. In the first step, where $t \in [0, 0.1]$, RE is about 0.76%. However, PT-PINN fails in the second step ($t \in [0, 0.2]$), with RE reaching 684.9%. Note that the architecture of the neural network in PT-PINN is a residual neural network, as used by Guo et al. [24]. Interestingly, when using a DNN (as in our strategy), PT-PINN fails to produce an acceptable solution even in the pretraining step ($t = 0.1$) for the case of $\beta = 20$ and $T = 1$. Therefore, our PP-PINN demonstrates superior performance for the convection equation.

As previously mentioned, the preconditioning strategy significantly improve the performance of PINN based methods. To provide a comprehensive analysis, we also evaluate the performance of the vanilla-PINN with preconditioning approach (PC-PINN). Specifically, we apply PC-PINN to solve the same task. For such a system, RE is approximately 100%. Consequently, the pretraining method is necessary for solving PDEs with high frequency solutions.

To further enhance the performance of PP-PINN, we implement the FP constraint. Comparing REs shown in Tables 1 and 2, we observe: (1) FP slightly decreases (increases) the accuracy (computation cost) of PP-PINN when $\hat{T}_k \leq 3$; (2) for $\hat{T}_k \geq 4$, FP significantly improves the accuracy of PP-PINN, similar to results observed in BC-PINN [22]. Therefore, for the convection equation, FP should be included when $\hat{T}_k$ is sufficiently large.

We also plot the predicted solution in the entire time domain with and without FP, along with the corresponding loss function in Fig. 5. In the upper panel of Fig. 5, where the FP constraint is absent, PP-PINN fails to provide an accurate solution when $t > 0.7$ ($\hat{t} > 7$), consistent with Table 1. Conversely, PP-PINN performs well in the presence of FP, as shown in the lower panel of Fig. 5 and Table 2. In both panels, loss functions converge to fixed values, indicating the completion of the optimization process. Therefore, to achieve high accuracy solutions with PP-PINN, the FP constraint is essential.

4.2. The reaction equation

In this subsection, we solve the reaction equation with a large reaction coefficient using PP-PINN. A typical unsolved reaction equation with periodic boundary conditions is given by

$$\frac{\partial u}{\partial t} - \rho u(1 - u) = 0, \quad (16)$$

$$u(x, 0) = h(x), \quad (17)$$

$$u(0, t) = u(2\pi, t), \quad (18)$$

where $x \in [0, 2\pi]$, $t \in [0, 1]$, ρ is the reaction coefficient, and $h = h(x)$ is the initial condition. As shown in Ref. [24], the exact solution of Eq. (16) is

$$u(x, t) = \frac{h(x) \exp(\rho t)}{h(x) \exp(\rho t) + 1 - h(x)}. \quad (19)$$

To facilitate comparison with the previous work [24], the initial condition is set as

$$h(x) = \alpha \exp\left(-\frac{(x - \pi)^2}{\epsilon \pi^2}\right), \quad (20)$$

where α and $\sqrt{\epsilon \pi^2}$ represent the amplitude and half-width of h .

Similar to Sec. 4.1, we first rewrite Eq. (16) as

$$\frac{\partial u}{\partial t} - c \rho u(1 - u) = 0, \quad (21)$$

Table 1

RE and computation times (CT) for the convection equation using PP-PINN with different $\hat{T}_k$. Hyperparameters used here are the same as in Fig. 2.

$\hat{T}_k$	1	2	3	4	5	6	7	8	9	10
RE (%)	0.17	0.22	0.42	0.99	1.41	1.58	1.87	27.62	40.65	50.29
CT (min)	3.61	9.53	15.63	28.13	41.05	55.34	73.85	105.41	139.02	177.81

Table 2

RE and CT for the convection equation using PP-PINN with different $\hat{T}_k$. Unlike Table 1, the FP constraint is implemented. Hyperparameters used here are the same as in Table 1.

$\hat{T}_k$	1	2	3	4	5	6	7	8	9	10
RE (%)	0.17	0.36	0.45	0.35	0.74	1.01	1.01	1.05	1.08	1.11
CT (min)	3.61	9.63	16.30	24.93	40.71	54.41	71.01	100.68	129.97	159.77

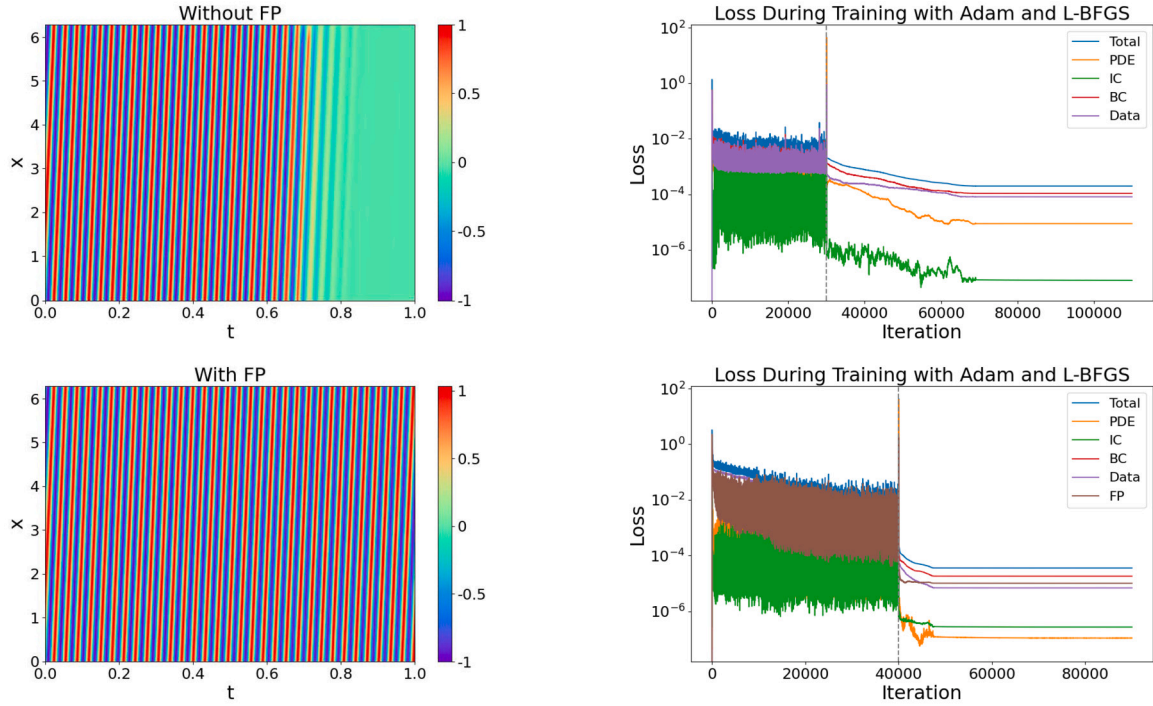


Fig. 5. The predicted solution and corresponding loss function without (upper) and with (lower) FP. Parameters used here are the same as in Table 2.

where $t = \hat{c}i$. The “effective reaction coefficient” is then expressed as $\rho^* = c\rho$. This transformation enables the pretraining step if c is appropriately chosen, as discussed in Sec. 4.1.

Next, we solve the transformed equation (Eq. (21)) with $\alpha = 1$ using PT-PINN, PT-PINN + FP, PC-PINN, and PP-PINN + FP (ours), respectively. First, we test the case with $\varepsilon = 0.2$. RE and CT of these methods are listed in Table 3. Generally, except for the case of PC-PINN, the reaction equation with these parameters can be solved by all other three approaches, which illustrates the necessity of the pretraining strategy for solving the reaction equation with $\varepsilon = 0.2$. The FP constraint slightly improves the performance of PT-PINN. Compared to PT-PINN, our approach enhances the accuracy of PINN by approximately three times, while the computation costs are comparable.

Notes that the computational cost refers to the time required for training over a specified number of epochs. In both PT-PINN and PP-PINN, pretrained solutions serve as suitable initializations and provide data constraints for the subsequent formal training step. This significantly reduces the training time for a given number of epochs. Consequently, despite involving more data points and a longer time domain during the formal training step, the computational cost can be lower than that of the pretraining step.

Table 3

RE and CT for the reaction equation using PT-PINN, PT-PINN + FP, PC-PINN, and PP-PINN + FP (ours). Here, $\rho = 20$, $\varepsilon = 0.2$, $c = 0.05$. The activation function is tanh; the numbers of hidden layers and neurons are 7 and 70; the sampling points are $N_i = N_j = 1000$.

Method	Pre-training ($t \in [0, 0.05]$)		Formal ($t \in [0, 1]$)	
PT-PINN	4.01×10^{-3}	5.39 min	7.68×10^{-3}	2.33 min
PT-PINN+FP	4.01×10^{-3}	5.39 min	6.80×10^{-3}	2.85 min
PC-PINN	—	—	9.96×10^{-1}	5.23 min
OURS	2.93×10^{-3}	3.22 min	2.31×10^{-3}	3.42 min

We also apply these four strategies to solve the reaction equation with $\varepsilon = 1$. Unlike the case of $\varepsilon = 0.2$, when we include the FP constraint, the accuracy of PT-PINN improves significantly (by about 20 times). Simultaneously, the reaction equation with $\varepsilon = 1$ can be directly and accurately solved using PC-PINN, achieving an accuracy of approximately 0.07%. Notably, PC-PINN demonstrates markedly superior accuracy compared to PT-PINN. Therefore, in this specific case, the preconditioning approach proves more effective than the pretraining strategy, contrasting with the situation observed when $\varepsilon = 0.2$. This dif-

Table 4

RE and CT for the reaction equation using PT-PINN, PT-PINN + FP, PC-PINN, and PP-PINN + FP (ours). Here, $\varepsilon = 1$. Other parameters are the same as in Table 3.

Method	Pre-training ($t \in [0, 0.05]$)		Formal ($t \in [0, 1]$)	
PT-PINN	2.02×10^{-3}	3.05 min	2.45×10^{-2}	9.58 min
PT-PINN+FP	2.02×10^{-3}	3.05 min	1.44×10^{-3}	8.13 min
PC-PINN	—	—	7.09×10^{-4}	4.16 min
OURS	5.71×10^{-4}	3.38 min	3.2×10^{-4}	2.28 min

ference can be attributed to the rapid transition of the variable u from its initial condition to the saturated state, as shown in the lower panel of Fig. 6. However, the efficacy of PT-PINN depends on the assumption that the PDE solution is smooth [24]. Conversely, appropriate preconditioning alleviates the high frequency and strong nonlinearity inherent in the reaction equation. Consequently, the performance of PC-PINN significantly surpasses that of PT-PINN. Additionally, our approach effectively enhances the performance of PINN (about three times better than PT-PINN + FP and two times better than PC-PINN). Furthermore, our method's computation cost is larger than PC-PINN while dramatically lower than the other two strategies which incorporate with the pretraining technique.

In the case of $\varepsilon = 0.2$, RE for all three approaches which incorporate the pretraining strategy is below 1%. Thus, the predicted solutions are consistent with the analytical solution. However, when $\varepsilon = 1$, the predicted solution obtained from PT-PINN shows about 2% error, while the other three show about 0.1%, 0.07%, and 0.03%. As shown in Fig. 6, the predicted solution from PT-PINN deviates from the analytical solution in the regimes where $x \sim 0$ and $t \sim 0$. However, this deviation disappears with the FP constraint. Furthermore, RE can be further reduced by adopting PP-PINN with FP, consistent with Table 4. In addition, for the case of $\varepsilon = 1$, the time derivatives of u approach infinity as $t \sim 0$, unlike the case of $\varepsilon = 0.2$. This may explain the significance of the FP constraint in these calculations.

We now provide some remarks on the performance of PC-PINN. Clearly, the computational cost of PC-PINN is significantly lower than that of approaches which incorporate the pretraining strategy. However, without pretraining, PC-PINN may encounter difficulties, as shown in Table 3. In fact, even for the cases which can be directly solved using such an approach, the accuracy of PC-PINN is lower than our PP-PINN (please see Tables 4 and 6). Therefore, combining preconditioning and pretraining strategies is crucial for enhancing the performance of PINN based methods in complex scenarios.

Unlike the convection equation discussed in Sec. 4.1, the reaction equation solved here is a nonlinear system. Therefore, the amplitude of u remains significant in the transformed equation even when the original task is appropriately normalized. Consequently, the amplitude effect should be studied more carefully.

Now let's discuss the performance of PP-PINN with different α . By defining $u = \alpha \hat{u}$, the reaction equation can be rearranged to

$$\frac{\partial \hat{u}}{\partial \hat{t}} - c\rho \hat{u}(1 - \alpha \hat{u}) = 0, \quad (22)$$

$$\hat{u}(x, 0) = \hat{h} = \frac{h(x)}{\alpha}, \quad (23)$$

$$\hat{u}(0, \hat{t}) = \hat{u}(2\pi, \hat{t}). \quad (24)$$

Here, $c\rho = 1$ is adopted. Since $\rho = 20$ is used in our calculations, $\hat{t} \in [0, \frac{1}{c}] = [0, 0.05]$. For the case of $\alpha = 10$, the pretraining step and formal step are set as $\hat{t} \in [0, 1]$ and $\hat{t} \in [0, 20]$, respectively. From the upper panel of Fig. 7, it is evident that Eq. (22) can be solved by our PP-PINN with credible accuracy (RE is about 2.56%). Unsurprisingly, the vanilla-PINN fails here (RE is about 118%). More interestingly, PT-PINN also fails to solve the original reaction equation (Eq. (16)) with $\alpha = 10$. Therefore, appropriate normalization schemes should be introduced if

Table 5

RE and CT for the heat equation using PP-PINN with $l = 0, 1, 2, 3$ and 4, respectively. Parameters used here are listed as follows. The activation function is sin, the numbers of hidden layers and neurons are 7 and 70, the sampling points are $N_i = N_j = 5000$, respectively.

l	Pre-training ($t \in [0, 0.2]$)		Formal ($t \in [0, 1]$)	
0	5.66×10^{-5}	4.56 min	8.63×10^{-5}	6.70 min
1	2.84×10^{-4}	4.07 min	8.64×10^{-5}	6.56 min
2	1.26×10^{-4}	4.01 min	1.01×10^{-4}	6.07 min
3	1.55×10^{-4}	4.12 min	2.76×10^{-4}	12.45 min
4	1.29×10^{-4}	7.24 min	4.53×10^{-4}	10.06 min

PINN and its variants are used to solve a system with solutions far from unity.

Usually, one may expect that the transformed reaction equation (Eq. (22)) with $\alpha = 0.1$ would be easier to solve than the one with $\alpha = 10$. This expectation is reasonable for a nonlinear system. However, our calculations do not support this hypothesis. For $\alpha = 0.1$, achieving a good prediction requires splitting the entire time domain into $\hat{t} \in [0, 1]$, $\hat{t} \in [0, 5]$, $\hat{t} \in [0, 15]$, and $\hat{t} \in [0, 20]$. With this operation, RE can reach 1.27%. This can be explained as follows: as shown in Fig. 7, with increasing t , u decreases sharply for $\alpha = 10$. For slightly larger t , the solution of the reaction equation remains almost flat. However, for $\alpha = 0.1$, u increases with t in the region where $t < 0.5$. Therefore, to obtain a high-accuracy solution, the entire time domain should be split more carefully. Note that both the vanilla-PINN and PT-PINN fail to solve Eq. (22) with $\alpha = 0.1$, further emphasizing the need for appropriate preconditioning strategies for PINN and its variants.

For comprehensiveness, we also apply PC-PINN to solve the aforementioned cases. As illustrated in Fig. 7, PC-PINN fails to solve these systems. Specifically, RE of these two cases are 78.52% and 99.99%, respectively. In fact, PC-PINN cannot predict such a system even when $\alpha = 1$ (please see also in Table 3).

4.3. The heat equation

To further illustrate the performance of PP-PINN, we solve the heat equation with strongly nonlinear diffusion coefficients. The nonlinear heat equation is written as follows

$$\frac{\partial u}{\partial t} - \nabla \cdot (u^l \nabla u) = Q(x, t), \quad (25)$$

$$u(x, 0) = \alpha[1 + 2\sin(\pi x)], \quad (26)$$

$$u(0, t) = \alpha[1 + 2\sin(2\pi t)], \quad (27)$$

$$u(1, t) = \alpha[1 + 2\sin(\pi + 2\pi t)], \quad (28)$$

where $t \in [0, 1]$, $x \in [0, 1]$, α is the amplitude of the solution, and l represents the nonlinearity. For simplicity, the exact solution is set as $u(x, t) = \alpha[1 + 2\sin(\pi x + 2\pi t)]$ [24]. The source term Q can then be derived from the expression for u .

Similar to previous subsections, we introduce $t = c\hat{t}$ and $u = \alpha\hat{u}$, and rearrange Eqs. (25)-(28) to

$$\frac{\partial \hat{u}}{\partial \hat{t}} - c\alpha^l \nabla \cdot (\hat{u}^l \nabla \hat{u}) = c\hat{Q} = c\frac{Q(x, t)}{\alpha}, \quad (29)$$

$$\hat{u}(x, 0) = 1 + 2\sin(\pi x), \quad (30)$$

$$\hat{u}(0, \hat{t}) = 1 + 2\sin(2\pi \hat{t}), \quad (31)$$

$$\hat{u}(1, \hat{t}) = 1 + 2\sin(2\pi \hat{t} + \pi). \quad (32)$$

Clearly, the exact solution of Eq. (29) is $\hat{u} = 1 + 2\sin(\pi x + 2\pi t)$. First, we test the performance of PP-PINN with $\alpha = 1$ and $c = 0.2$, implementing the FP constraint as well. As shown in Table 5, the nonlinear heat equation (Eq. (29)) can be solved by PP-PINN with high accuracy and reasonable computation cost.

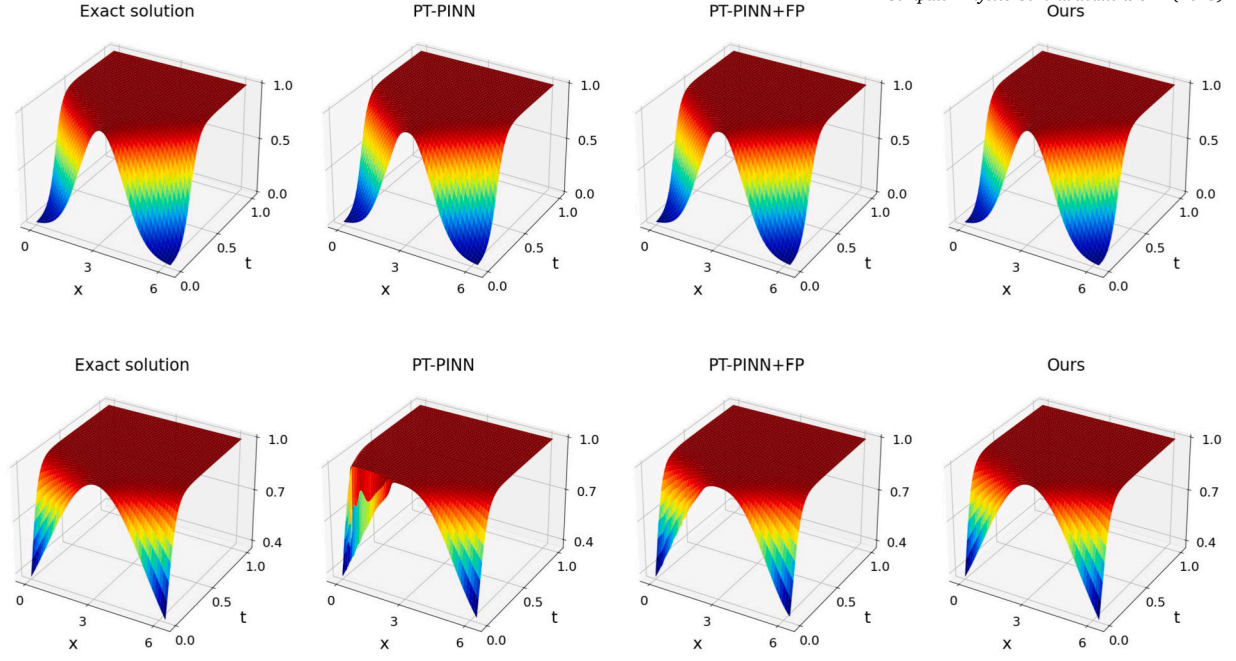


Fig. 6. Exact solution and predicted solution based on PT-PINN, PT-PINN + FP, and PP-PINN + FP (ours) for $\varepsilon = 0.2$ (upper) and $\varepsilon = 1$ (lower). Here, $\rho = 20$, $c = 0.05$, $\omega_{\text{PDE}} = \omega_{\text{IC}} = \omega_{\text{BC}} = \omega_{\text{DATA}} = \omega_{\text{FP}} = 1$. The activation function is tanh; the numbers of hidden layers and neurons are 7 and 70; the numbers of sampling points are $N_i = N_j = 1000$.

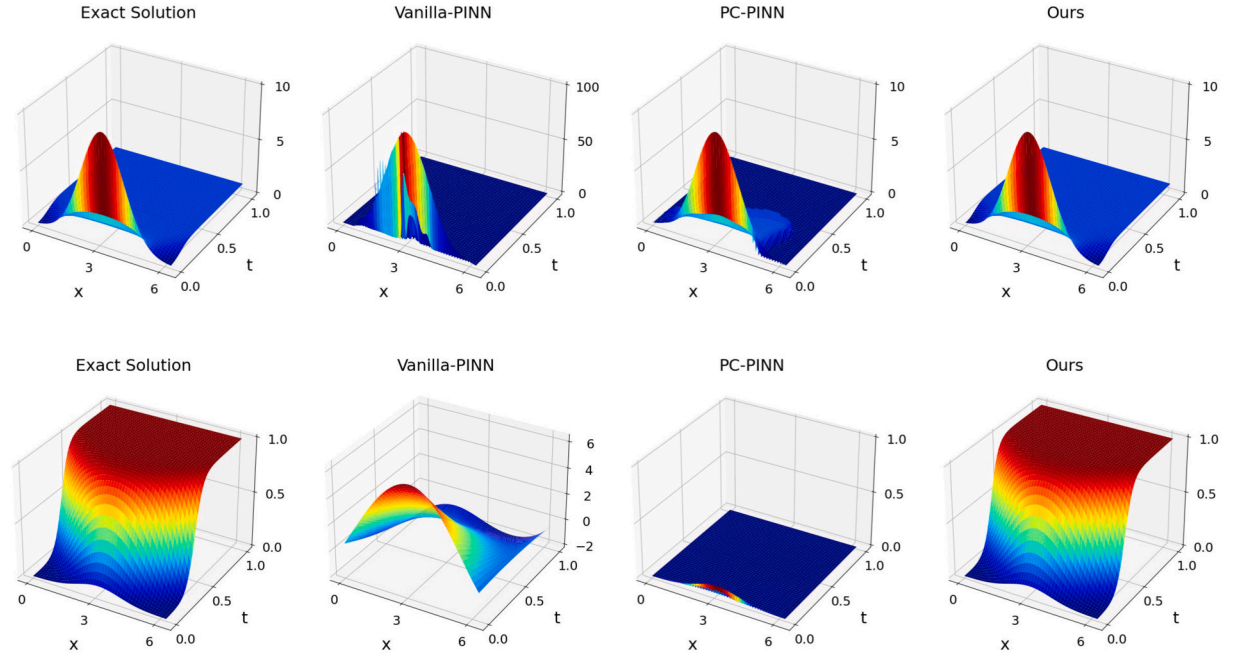


Fig. 7. Exact solution and predicted solution based on vanilla-PINN, PC-PINN, and PP-PINN (ours) for $\alpha = 10$ (upper) and $\alpha = 0.1$ (lower). Here, $\rho = 20$, $c = 0.05$, $\varepsilon = 0.2$. Hyperparameters used here are the same as in Fig. 6.

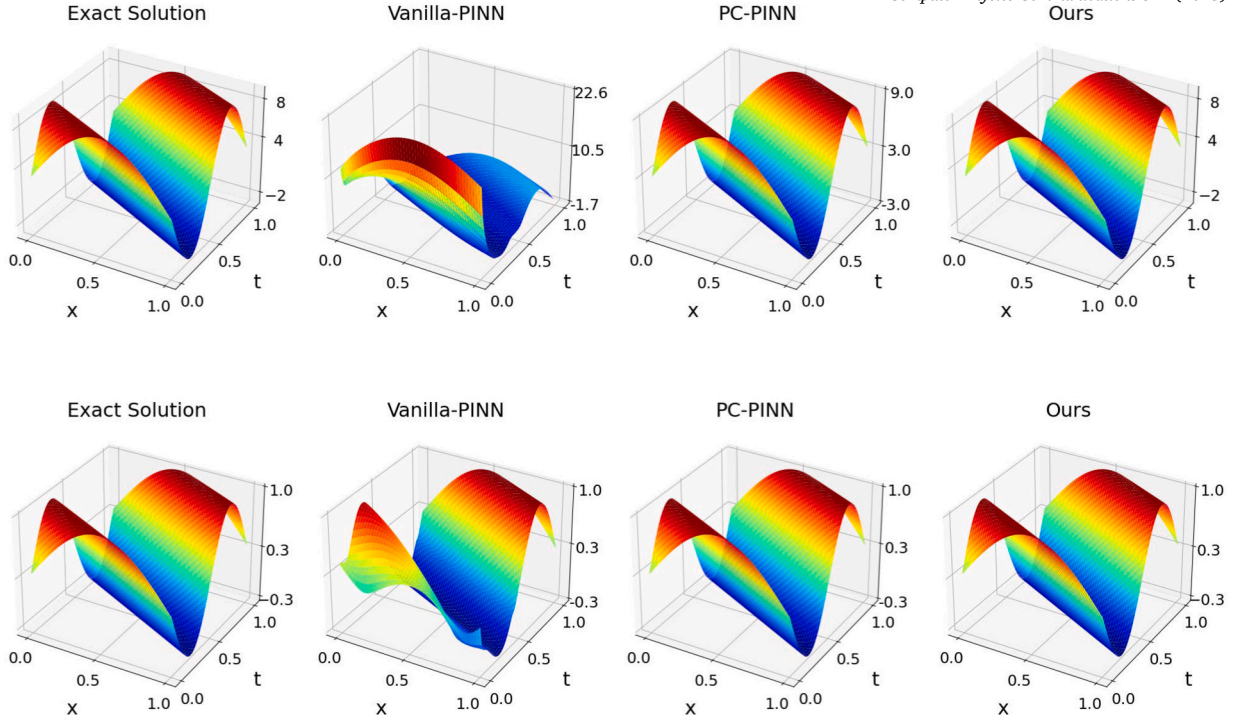


Fig. 8. Exact solution and predicted solution based on vanilla-PINN, PC-PINN, and PP-PINN (ours) for $\alpha = 3$ (upper) and $\alpha = \frac{1}{3}$ (lower). Hyperparameters used here are the same as in Table 5.

Table 6

RE and CT for the heat equation using PT-PINN, PT-PINN + FP, PC-PINN, and PP-PINN + FP (ours) with $l = 2$ and $l = 3$, respectively. Parameters used here are the same with Table 5.

Method	Pre-training		Formal	
PT-PINN ($l = 2$)	7.86×10^{-5}	8.01 min	1.49×10^{-4}	8.34 min
PT-PINN+FP ($l = 2$)	7.86×10^{-5}	8.01 min	1.30×10^{-4}	9.59 min
PC-PINN($l = 2$)	—	—	1.63×10^{-4}	7.33 min
OURS ($l = 2$)	1.26×10^{-4}	4.01 min	1.01×10^{-4}	6.07 min
PT-PINN ($l = 3$)	4.31×10^{-4}	11.90 min	4.19×10^{-4}	11.94 min
PT-PINN+FP ($l = 3$)	4.31×10^{-4}	11.90 min	3.07×10^{-4}	9.37 min
PC-PINN($l = 3$)	—	—	3.82×10^{-4}	14.98 min
OURS ($l = 3$)	1.55×10^{-4}	4.12 min	2.76×10^{-4}	12.45 min

To compare with the previous work [24], we also list RE and CT of four different strategies in Table 6 for $l = 2$ (weak nonlinearity) and $l = 3$ (strong nonlinearity). It appears that the nonlinear heat equation can be solved by all four methods. However, our approach demonstrates better accuracy. Among the three approaches that incorporate the pretraining strategy, our method shows the lowest computational cost. Additionally, the extra FP constraint improves the performance of PINN in both weak ($l = 2$) and strong ($l = 3$) nonlinearity cases.

Based on the above studies, we further solve Eq. (29) with $\alpha = 3$ and $\alpha = \frac{1}{3}$, adopting $l = 2$. From Fig. 8, it can be observed that for both cases of $\alpha = 3$ and $\alpha = \frac{1}{3}$, PP-PINN can predict the nonlinear heat equation with high accuracy (0.02% and 0.01%, respectively). However, the vanilla-PINN fails to solve the original task (Eq. (25)) with $\alpha = 3$ and $\alpha = \frac{1}{3}$. In fact, REs for these two cases are 57.40% and 28.30%, respectively. Additionally, Eq. (25) with $\alpha = 3$ and $\alpha = \frac{1}{3}$ cannot be solved by PT-PINN with credible accuracy. As discussed earlier, appropriate preconditioning schemes should be introduced when solving PDEs with PINN and its variants. In fact, such a system can be directly solved using PC-PINN with high accuracy (0.03% and 0.02% for cases of $\alpha = 3$ and $\alpha = \frac{1}{3}$, respectively), although the precision is slightly lower than our approach.

Now we give some remarks on the case of $\alpha = 3$. To advance our investigation, we solve this task based on two distinct variants of PP-PINN. The first variant employs the original formulation while the second variant incorporates solely include the data constraint obtained from the pretraining step. Numerical results show that RE of these two variants are 1.29% and 0.02%, respectively. The observed phenomenon can may be attributed to the large derivatives of the variable u , which are proportional to α . Typically, it is anticipated that transfer learning, i.e., the initialization scheme, will accelerate the convergence during formal training [24]. However, when $\alpha \sim 3$, the initialization scheme results in a negative transfer effect in transfer learning. This aspect requires further investigation in future research.

5. Summary and discussion

In this work, we propose a new variant of PINN, named PP-PINN. The main idea originates from several observations. It is known that the vanilla-PINN struggles to solve PDEs with high-frequency solutions or strong nonlinearity. Additionally, previous studies typically set the amplitudes of unsolved equations (α) in the order of $\mathcal{O}(1)$. However, this setting may not be appropriate for realistic studies [27,28]. When the amplitude is much larger than unity, the nonlinearity becomes stronger, which can cause the training procedure of PINN to fail. In fact, we found that for cases where $\alpha \ll 1$ or $\alpha \gg 1$, the vanilla-PINN fails to predict credible solutions. To address these issues, we propose a preconditioning method that transforms the original task into a new system with low-frequency solutions and weak nonlinearity over a longer timescale. However, long-time integration via PINN may violate causality, which is another reason why PINN fails to predict high-accuracy solutions [20]. Fortunately, this issue can be resolved using the pretraining strategy, which splits the entire time domain into a series of sub-domains.

Numerical experiments demonstrate that when schemes of normalization and domain decomposition are appropriately chosen (as seen in Secs. 4.1, 4.2, and 4.3), the transformed system can be stably solved in the first sub-domain. Similar to previous work [24], solutions obtained in sub-domains provide initialization and additional data constraints for

relatively large sub-domains. By combining these with the pretraining strategy, we can solve the new system in sub-domains sequentially.

It should be noted that our approach faces challenges with cases involving extremely long timescales. For example, in Sec. 4.1, we found that PP-PINN fails to predict solutions when $\hat{T} > 7$. Mathematically, this suggests that it is difficult to solve the convection equation with $\beta > 140$ using such a deep learning-based method. To further improve the performance of our PP-PINN, we introduced a new data constraint, the “fixed point” (FP). It was observed that when this constraint is applied, our PP-PINN provides credible solutions for the convection equation with $\beta = 20$ and $\hat{T} = 10$. Therefore, the FP constraint is essential for cases with extremely high-frequency solutions or strong nonlinearity.

In all illustrated situations, our PP-PINN demonstrates better performance than previous methods. For instance, the convection equation with $\beta = 20$ and $\hat{T} = 10$, equivalent to the case with $\beta = 200$ and $T = 1$, can be stably predicted using PP-PINN. In previous work, credible solutions were only obtainable when $\beta < 60$ and $T = 1$, which is significantly less than what our algorithm can achieve. For nonlinear systems, as discussed in Secs. 4.2 and 4.3, the performance of PP-PINN is on par with other PINN-based methods for weak nonlinearity cases. However, when nonlinearity is sufficiently strong, especially in regimes where $\alpha \gg 1$, the predictive capability of our strategy is markedly superior to previous methods.

In summary, we propose the PP-PINN strategy for predicting PDEs with high-frequency solutions and strong nonlinearity. This approach offering improved accuracy and reduced computational costs compared to the previous method [24]. Additionally, the “fixed point” (FP) constraint is introduced to enhance performance in extreme cases. Furthermore, we address the underexplored issue of α 's impact on PINN performance and resolve it with the preconditioning strategy. However, to achieve higher accuracy predictions necessary for realistic studies in natural science [29–31], our method requires further improvement. In the future, we plan to combine recently developed frameworks and optimizers to enhance the performance of PINN [32–37].

CRedit authorship contribution statement

Zihan Wang: Writing – original draft, Validation, Software, Investigation, Formal analysis, Data curation, Conceptualization. **Ziyue Hu:** Software, Investigation, Formal analysis, Data curation. **Mingwei Yang:** Validation, Software, Investigation. **Yalin Dong:** Writing – review & editing, Investigation. **Wenlong Huang:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Investigation, Data curation, Conceptualization. **Haijun Ren:** Writing – review & editing, Supervision, Project administration, Investigation, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Natural Science Foundation of China Grant No. 12205005 and 12175228, the Anhui Provincial Natural Science Foundation Grant No. 2108085QA34, the Natural Science Research

Project of Anhui Educational Committee Grant No. 2024AH040028, the Project of Anhui International Joint Research Center for Metallurgical Process and System Science, Anhui University of Technology No. 2024004.

Data availability

The software program and source codes can be downloaded from the following link: <https://github.com/huangwenlongplasma/PP-PINN>.

References

- [1] C. Beck, L. Gonon, A. Jentzen, arXiv:2003.00596, 2020.
- [2] C. Beck, F. Hornung, M. Hutzenthaler, A. Jentzen, T. Kruse, J. Numer. Math. 28 (2020) 197–222.
- [3] S. Becker, R. Braunwarth, M. Hutzenthaler, A. Jentzen, P. von Wurstemberger, arXiv: 2005.10206, 2020.
- [4] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, S.F. Wang, L. Yang, Nat. Rev. Phys. 3 (2020) 422–440.
- [5] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Nat. Mach. Intell. 3 (2021) 218–229.
- [6] M. Raissi, P. Perdikaris, G. Karniadakis, J. Comput. Phys. 378 (2019) 686–707.
- [7] W.N. E, B. Yu, Commun. Math. Stat. 6 (2018) 1–12.
- [8] J. Sirignano, K. Spiliopoulos, J. Comput. Phys. 375 (2018) 1339–1364.
- [9] Z.Y. Hu, K. Shukla, G.E. Karniadakis, K. Kawaguchi, Neural Netw. 176 (2024) 106369.
- [10] L. Yuan, Y.Q. Ni, X.Y. Deng, S. Hao, J. Comput. Phys. 2022 (462) (2022) 111260.
- [11] P. Ren, C.P. Rao, J.X. Wang, H. Sun, Comput. Methods Appl. Mech. Eng. 389 (2022) 114399.
- [12] X.H. Meng, Z. Li, D.K. Zhang, G.E. Karniadakis, Comput. Methods Appl. Mech. Eng. 2020 (370) (2024) 113250.
- [13] L.L. Zhong, Q. Gu, B.Y. Wu, Comput. Phys. Commun. 257 (2020) 107496.
- [14] J.F. Urbán, P. Stefanou, C. Dehman, J.A. Pons, Mon. Not. R. Astron. Soc. 524 (2023) 32–42.
- [15] H.V. Alvarez, G. Fabiani, N. Kazantzis, I.G. Kevrekidis, C. Siettos, Chaos Solitons Fractals 186 (2024) 115215.
- [16] J.W. Ye, W. Zeng, N.C. Do, M. Lambert, Water Res. 257 (2024) 121648.
- [17] R. Li, E. Lee, T.F. Luo, Phys. Rev. Appl. 19 (2023) 064049.
- [18] Y.L. Huang, J.H. Seinfeld, Environ. Sci. Technol. 56 (2022) 4676–4685.
- [19] H.T. Chen, E. Kätelhön, R.G. Compton, Anal. Chem. 95 (2023) 12826–12834.
- [20] S.F. Wang, S. Sankaran, P. Perdikaris, Comput. Methods Appl. Mech. Eng. 421 (2024) 116813.
- [21] Z.Q. Xu, Y.Y. Zhang, T. Luo, Y.Y. Xiao, Z. Ma, Commun. Comput. Phys. 5 (2020) 1746–1767.
- [22] R. Matthey, S. Ghosh, Comput. Methods Appl. Mech. Eng. 390 (2022) 114474.
- [23] S. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, M.W. Mahoney, Adv. Neural Inf. Process. Syst. 34 (2021) 26548–26560.
- [24] J.W. Guo, Y.Z. Yao, H. Wang, T.X. Gu, J. Comput. Phys. 489 (2023) 112258.
- [25] Hailong Sheng, Chao Yang, J. Comput. Phys. 428 (2021) 110085.
- [26] Lu Lu, Raphael Pestourie, Wenjie Yao, Zhicheng Wang, Francisc Verdugo, Steven G. Johnson, SIAM J. Sci. Comput. 43 (2021) B1105–B1132.
- [27] W.L. Huang, P. Zhu, Phys. Plasmas 61 (2015) 036047.
- [28] W.L. Huang, P. Zhu, Nucl. Fusion 22 (2021) 032502.
- [29] Yu Liu, Rong Jin, Minchi Li, Wenlong Huang, Xexin Huang, Junnan Jiang, Pengcheng Yu, Jiuhou Lei, Geophys. Res. Lett. 51 (2024) e2023GL107239.
- [30] Wenlong Huang, Yu Liu, Pengcheng Yu, Jiuhou Lei, Astrophys. J. 975 (2024) 157.
- [31] Junnan Jiang, Yu Liu, Wenlong Huang, Minchi Li, Rong Jin, Pengcheng Yu, Jiuhou Lei, Astrophys. J. 977 (2024) 142.
- [32] S. Rojas, P. Maczuga, J. Muñoz-Matute, D. Pardo, M. Paszyński, Comput. Methods Appl. Mech. Eng. 425 (2020) 116904.
- [33] J. Müller, M. Zeinhofer, in: Proceedings of the 40th International Conference on Machine Learning, vol. 202, 2023, pp. 25471–25485.
- [34] Y.Z. Wang, J. Sun, J.S. Bai, C. Anitescu, M.S. Eshaghi, X.Y. Zhuang, T. Rabczuk, Y.H. Liu, arXiv:2406.11045v1, 2024.
- [35] Yusuke Yamazaki, Ali Harandi, Mayu Muramatsu, Alexandre Viardin, Markus Apel, Tim Brepols, Stefanie Reese, Shahed Rezaei, Eng. Comput. (2024) 1–29.
- [36] Xuwei Zhou, Sheng Sun, Songlin Cai, Gongyu Chen, Honghui Wu, Jie Xiong, Jiaming Zhu, J. Mater. Sci. 59 (2024) 15727–15737.
- [37] Yuwei Geng, Yuankai Teng, Zhu Wang, Lili Ju, J. Comput. Phys. 496 (2024) 112589.